

Reviewer Pack — Algebraic K-Theory and Boolean Circuit Topological Entropy C...

Entry e3f5a8708c5c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 14:23:56 UTC

Algebraic K-Theory and Boolean Circuit Topological Entropy Correlation

Entry ID: e3f5a8708c5c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 14:23:56 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every boolean circuit C with n inputs, the topological entropy of its state-space representation, $H(C)$, is linearly correlated with the size of its minimal polynomial over the finite field $GF(2^m)$, where m is the depth of C .

Rationale (proposer's reasoning):

Algebraic K-theory offers a framework for studying the obstructions to certain kinds of homotopy equivalences, which can be used to represent boolean circuits. By linking this theory with topological entropy, we may uncover new insights into the complexity of boolean circuits and their structure.

Taxonomy category: c003b_cumulative_entropy/SC2#c49 (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0fa237dffe391a9a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$, the correlation coefficient r between topological entropy $H(C)$ and $\log(m)$ across 30 random seeds is $|r| \geq 0.7$. The conjecture is falsified if $|r| < 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Algebraic K-Theory' AND 'Boolean circuit topological entropy' - 'Minimal polynomial' AND 'finite field $GF(2^m)$ ' AND 'boolean circuit complexity' - 'State-space representation' AND 'topological entropy' AND 'algebraic K-theory'

Top relevant hits considered: - [http://arxiv.org/abs/1601.06285v3] Twisted iterated algebraic K-theory and topological T-duality for sphere bundles - [http://arxiv.org/abs/0903.3983v1] Algebraic v. topological K-theory: a friendly match - [http://arxiv.org/abs/2001.01608v2] The plethora of operations in complex topological K-theory - [http://arxiv.org/abs/1612.04588v1] Reverse Engineering of Irreducible Polynomials in $GF(2^m)$ Arithmetic - [http://arxiv.org/abs/1208.5345v2] Generating All Minimal Edge Dominating Sets with Incremental-Polynomial Delay - [http://arxiv.org/abs/1102.5335v1] Block Companion Singer Cycles, Primitive Recursive Vector Sequences, and Coprime Polynomial Pairs over Finite Fields - [http://arxiv.org/abs/2107.09581v2] Entropy as a Topological Operad Derivation - [http://arxiv.org/abs/cs/0611047v1] The Reaction RuleML Classification of the Event / Action / State Processing and Reasoning Space

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_circuit(n):
    if n == 1:
        return random.choice([0, 1])
    else:
        left = generate_random_circuit(n // 2)
        right = generate_random_circuit(n - n // 2)
        return (left, right)

def state_space_representation(circuit):
    if isinstance(circuit, int):
        return {circuit}
    else:
        left_states = state_space_representation(circuit[0])
        right_states = state_space_representation(circuit[1])
        return {x ^ y for x in left_states for y in right_states}
```

```

def minimal_polynomial(state_space, m):
    if not state_space:
        return 0
    poly = [1]
    for s in state_space:
        new_poly = []
        for coeff in poly:
            new_poly.append(coeff * s)
        poly.extend(new_poly)
    return poly

def topological_entropy(state_space):
    n = len(state_space)
    if n == 0:
        return 0
    log_n = math.log2(n)
    entropy = -sum(1 / n * math.log2(1 / n) for _ in range(n))
    return entropy

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        instances_tested = 0
        total_entropy = 0
        total_log_m = 0
        for _ in range(30):
            circuit = generate_random_circuit(n)
            state_space = state_space_representation(circuit)
            m = len(state_space) - 1
            if m == 0:
                continue
            poly = minimal_polynomial(state_space, m)
            entropy = topological_entropy(state_space)
            total_entropy += entropy
            total_log_m += math.log2(m)
            instances_tested += 1
        if instances_tested < 30:
            return {
                "metric_name": "correlation_coefficient",
                "metric_value": None,
                "instances_tested": instances_tested,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "not_enough_instances"
            }
        mean_entropy = total_entropy / instances_tested
        mean_log_m = total_log_m / instances_tested
        covariance = sum((entropy - mean_entropy) * (math.log2(m) - mean_log_m) for entropy, m in
            zip(results, range(5, 41))) / instances_tested
        variance_m = sum((math.log2(m) - mean_log_m) ** 2 for m in range(5, 41)) / instances_tested
        correlation_coefficient = covariance / math.sqrt(variance_m * instances_tested)
        results.append(correlation_coefficient)
    return {
        "metric_name": "correlation_coefficient",

```


skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the required computations to evaluate the conjecture. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17598
2	propose	ollama_remote	glm4:latest	0	0	17039
3	preregistration	ollama_remote	glm4:latest	0	0	9170
4	novelty	ollama_remote	glm4:latest	0	0	8519
5	novelty	ollama_remote	glm4:latest	0	0	12395
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22617
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22190
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29488
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17913
10	judge	ollama_remote	glm4:latest	0	0	9145

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 166075 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/e3f5a8708c5c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e3f5a8708c5c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e3f5a8708c5c.tar.gz (if generated)